Extending PRAC Framework to Graph Algorithms: Breadth-First Search (BFS)

Hitarth Makawana
Roll No: 230479
hitarthkm23@iitk.ac.in

Ronav Puri
Roll No: 230815
ronavgp23@iitk.ac.in

April 15, 2026

1 Introduction

Breadth-First Search (BFS) is a fundamental graph traversal algorithm. However, executing BFS in a Secure Multi-Party Computation (MPC) setting introduces severe privacy risks, primarily **topology leakage**. *Standard pointer-chasing* and *dynamic queue structures* inherently leak the degrees of individual vertices and the overall diameter of the graph through memory access patterns.

This report details a novel **Oblivious Breadth-First Search (OBFS) protocol** tailored for the **PRAC (Private Random Access Computations) 3-party framework**. By flattening the graph representation and replacing dynamic pointer-chasing with data-independent Oblivious Bitonic Sorts and linear scans, this protocol guarantees zero topology leakage while optimizing for network round efficiency.

2 Data Structure: The Graph Array

To prevent degree leakage without incurring the massive memory overhead of dummy-edge padding (where every node is padded to a maximum degree D_{max}), the graph is stored as a tightly packed, single-dimensional array.

The graph is represented as a secret-shared Distributed ORAM (DORAM) array G of size $N = V + E$. There is no physical separation between vertex records and edge records. Each element $G[i]$ is a secret-shared struct containing the following fields, where the computing parties hold either additive shares (AS) or boolean shares (BS):

- $isEdge^{BS} = 1$ (if struct corresponds to an edge)
  $= 0$ (if struct corresponds to a vertex)
- v_1^{AS}(Source vertex) $= i$ (if $isEdge = 0$ and vertex no. $= i$)
  $= i$ (if $isEdge = 1$ and edge $= (i, j)$)
- v_2^{AS}(Destination vertex) $= i$ (if $isEdge = 0$ and vertex no. $= i$)
  $= j$ (if $isEdge = 1$ and edge $= (i, j)$)
- q^{BS}(Queue flag) $= 1$ (if the vertex/edge is currently being explored or part of the queue)
  $= 0$ (otherwise)
- $visited^{BS} = 1$ (if the vertex/edge was placed in the queue in current or any previous step)
  $= 0$ (otherwise)
- key^{AS} (Transient sorting key used strictly during the routing phases)

3 Cryptographic Primitives

The protocol leverages the following highly optimized PRAC primitives to bypass the need for heavy DUORAM index masking:

1. **MPC AND/OR:** Parties P_0 and P_1 hold boolean shares of $x \in 0,1$ and $y \in 0,1$, and wish to jointly compute boolean shares of the AND of x and y. We denote the operation as $z^{BS} \leftarrow x^{BS} \wedge y^{BS}$.
   (MPC OR can be realized using the MPC AND primitive, since $A \vee B = A + B - A \wedge B$)
2. **MPC Flag-Word (FW) Multiplication:** P_0 and P_1 hold additive shares of $y \in \mathbb{Z}_{2^r}$ and boolean shares of $f \in 0,1$. The two parties compute additive shares of $z \in \mathbb{Z}_{2^r}$, such that $z = f \cdot y$. We denote this as $z^{AS} \leftarrow f^{BS} \cdot y^{AS}$.
3. **Oblivious Select:** P_0 and P_1 hold additive shares of both x and y. Oblivious Select, denoted as OSelect, is defined as $z^{AS} \leftarrow \text{OSelect}(x^{AS}, y^{AS}, b^{BS})$. If $b = 1$, then $z = y$ and if $b = 0$, then $z = x$.
4. **Oblivious Bitonic Sort:** PRAC provides an implementation of bitonic sort, which will obliviously sort N items in $\mathcal{O}(\log^2 N)$ rounds.

4 The PRAC - Secure BFS Protocol

To completely hide the graph's depth, the outer BFS loop executes a strict worst-case bound of $V - 1$ iterations. Each iteration consists of two main phases that route the *queue status* through the graph without relying on pointers.

4.1 Initialization

The graph is loaded into G. For the starting (source) vertex S:

- $q^{BS} = 1$
- $visited^{BS} = 1$

For all other vertices and all edges, $q^{BS} = 0$, and $visited^{BS} = 0$.

4.2 Phase 1: Source-to-Edge Propagation

The goal is to pass the queue flag from vertices to their outgoing edges.

1. **Prepare Sort Key:** For every element $i \in [1, N]$:

$$G[i].key^{AS} \leftarrow G[i].v_1^{AS}$$

2. **Oblivious Sort:** The array G is sorted using oblivious bitonic sort provided by PRAC.
   - *Primary Criterion:* key^{AS} (ascending)
   - *Secondary Criterion:* $isEdge^{BS}$ (ascending)

   Result: Every vertex struct is immediately followed by its outgoing edge structs.
3. **Forward Scan (Trickle-Down):** A linear scan propagates the queue flag using a secret-shared running register t_1^{BS}, initialized to 0. The intuition is that *if a vertex is currently in the queue, all its outgoing edges must also be inserted in the queue.*

Algorithm 1 Phase 1: Source-to-Edge Propagation (Trickle-Down)

Input: Graph array G of size N $(= V + E)$

for $i = 1$ **to** N **do**
  $G[i].key^{AS} \leftarrow G[i].v_1^{AS}$
end for

OSort(G, key, $isEdge$)

$t_1^{BS} \leftarrow 0$
for $i = 1$ **to** N **do**
  $t_1 \leftarrow$ OSelect($G[i].q$, t_1, $G[i].isEdge$)
  $G[i].q \leftarrow$ OSelect($G[i].q$, t_1, $G[i].isEdge$)
end for

4.3 Phase 2: Edge-to-Destination Resolution

The goal is to pass the queue status from the edges to their destination vertices to form the next iteration's queue, resolving write-collisions obliviously.

1. **Prepare Sort Key:** For every element $i \in [1, N]$:

$$G[i].key^{AS} \leftarrow G[i].v_2^{AS}$$

2. **Oblivious Sort:** The array G is sorted again using oblivious bitonic sort provided by PRAC.
   - *Primary Criterion:* key^{AS} (ascending)
   - *Secondary Criterion:* $type^{BS}$ (descending)

   Result: All incoming edge structs appear immediately before their corresponding destination vertex struct.

3. **Forward Scan (Accumulate and Apply):** A linear scan accumulates active edges and activates unvisited target vertices using another secret-shared running registers t_2^{BS} and t_3^{BS}, initialized to 0. The intuition is that *if any one of the incoming edges is currently in the queue, the destination vertex must be inserted in the queue.*

4. **Edge Cleanup:** Obliviously clear all edge frontier flags for the next iteration.

5 Complexity Analysis

- **Communication and Round Complexity:** The bottleneck is the bitonic sort, which requires $\mathcal{O}(\log^2(V+E))$ rounds. With two sorts per level across $V-1$ levels, the total round complexity is strictly bounded by $\mathcal{O}(V \cdot \log^2(V+E))$. The local computation and online bandwidth are exceptionally light and strictly linear, i.e., $\mathcal{O}(V+E)$ per phase, completely independent of the graph's actual density.
- **Security:** Topology is perfectly hidden. Access patterns depend strictly on public indices (sorting networks and linear increments 1 to N), preventing any degree or depth leakage.

6 Optional Optimizations to Eliminate Online Sorting

While the baseline OBFS protocol relies on Oblivious Bitonic Sort to dynamically route queue flags without pointer-chasing, the sorting network introduces a polylogarithmic $\mathcal{O}(\log^2 N)$ round and bandwidth overhead per iteration. Since the graph topology is static during a BFS traversal, we propose two theoretical optimizations to entirely eliminate sorting in the online phase.

Algorithm 2 Phase 2: Edge-to-Destination Resolution

Input: Graph array G of size N $(= V + E)$

for $i = 1$ **to** N **do**
  $G[i].key^{AS} \leftarrow G[i].v_2^{AS}$
end for

OSort(G, key, $isEdge$)

$t_2^{BS} \leftarrow 0$, $t_3^{BS} \leftarrow 0$
for $i = 1$ **to** N **do**
  $t_2^{BS} \leftarrow$ OSelect(t_2^{BS}, $t_2^{BS} \vee G[i].q^{BS}$, $G[i].isEdge^{BS}$)
  $t_3^{BS} \leftarrow$ OSelect($t_2^{BS} \wedge \neg G[i].visited^{BS}$, t_3^{BS}, $G[i].isEdge^{BS}$)
  $G[i].q^{BS} \leftarrow$ OSelect(t_3^{BS}, $G[i].q^{BS}$, $G[i].isEdge^{BS}$)
  $G[i].visited^{BS} \leftarrow$ OSelect($G[i].visited^{BS} \vee t_3^{BS}$, $G[i].visited^{BS}$, $G[i].isEdge^{BS}$)
  $t_2^{BS} \leftarrow$ OSelect(0, t_2, $G[i].isEdge$)
end for

for $i = 1$ **to** N **do**
  $G[i].q^{BS} \leftarrow$ OSelect($G[i].q^{BS}$, 0, $G[i].isEdge^{BS}$)
end for

6.1 Implicit Source Ordering and DORAM Pointer-Chasing

The first optimization relies on structurally initializing the graph array and using PRAC's underlying DUORAM for oblivious random access.

- **Eliminating Phase 1 sort:** In the preprocessing phase, the graph array G is initialized strictly in **source order**, i.e., each vertex is immediately followed by its outgoing edges. This static layout completely eliminates the need for the Phase 1 sort, allowing the source-to-edge propagation to occur via a single $\mathcal{O}(N)$ linear trickle-down scan.
- **Eliminating Phase 2 sort:** Instead of sorting incoming edges to sit alongside their destination vertices, each edge stores a direct DUORAM pointer to its destination vertex. The queue flag is passed by executing an oblivious memory update to the destination struct.
- **Complexity and Trade-offs:** To prevent topology leakage, a DUORAM update must be executed for *every* edge E in the graph during every iteration, regardless of its active state. While PRAC's DUORAM evaluates updates in $\mathcal{O}(1)$ online rounds, this approach requires generating $\mathcal{O}(|E|)$ Distributed Point Functions (DPFs) per level. Across $V - 1$ iterations, the preprocessing phase must generate $\mathcal{O}(V \cdot E)$ DPFs, exchanging the online round bottleneck for a massive offline bandwidth and storage overhead.

6.2 Pre-computed Permutations via Secure Shuffle

Taking inspiration from the Graphiti framework, we can exploit the static nature of the graph by replacing data-independent sorting with pre-computed permutations.

- **Mechanism:** In the offline phase, the parties compute the exact permutation $\pi_{S \to D}$ required to map the graph from **Source Order** to **Destination Order**, as well as the inverse permutation $\pi_{D \to S}$.
- **Online Execution:** During the online phase, the Oblivious Bitonic Sort is replaced by a constant-round Secure Shuffle. The parties apply the secret-shared permutation $\pi_{S \to D}$ to route flags from edges to destinations (Phase 2), and $\pi_{D \to S}$ to reset the array for the next level (Phase 1).

- **Complexity and Trade-offs:** A Secure Shuffle requires only $\mathcal{O}(1)$ rounds and strictly linear $\mathcal{O}(N)$ bandwidth. This strictly dominates both the baseline protocol and Optimization 1.

The table below illustrates the complexity shifts for a single BFS level across the baseline and proposed optimizations (where $N = V + E$):

Optimization	**Online Rounds**	**Online Bandwidth**	**Preprocessing Overhead**
No optimization	$\mathcal{O}(\log^2 N)$	$\mathcal{O}(N \log^2 N)$	Minimal (Standard MPC Triples)
Optimization 1	$\mathcal{O}(1)$	$\mathcal{O}(N)$	Massive ($\mathcal{O}(E)$ DPFs per level)
Optimization 2	$\mathcal{O}(1)$	$\mathcal{O}(N)$	Moderate (Permutation generation)